SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE**: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour QUESTIONS: 3 Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I ***D Dharshana(192524243)*** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

Student 9: Dharshana D | Register Number: 192524243

1. Scenario

An event management company is designing an AI system to automatically generate seating arrangements for a large wedding banquet. The system must satisfy multiple constraints: no two guests with known conflicts should be seated at the same table, a limited number of seats are available per table, certain family groups must be seated together, guests with dietary/accessibility needs require specific table locations, and special accommodations for VIP guests near the head table. The complexity increases as the number of guests and tables grows.

Tasks:

Formulate the problem as a Constraint Satisfaction Problem (CSP).

i. Clearly define variables, domains, and constraints with explanation.

ii. Explain how backtracking search works in this scenario.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

iv. Analyze real-world challenges and justify the best strategy for scalability.

AI-Based Wedding Seating Arrangement Using CSP

i. Formulation as a Constraint Satisfaction Problem (CSP)

A **Constraint Satisfaction Problem (CSP)** consists of **variables, domains, and constraints**.

Variables

Each guest can be considered a variable.

- Let represent the guests.
- The value assigned to each guest represents the table at which the guest will sit.

For example:

Domains

The domain of each guest contains the tables where that guest can potentially sit.

Example:

For guests with special requirements, the domain is restricted. For example, a wheelchair user may only have tables with suitable accessibility.

Constraints

1. **Conflict constraint:** Guests with known conflicts cannot sit at the same table.

for conflicting guests.

2. **Table capacity:** Each table can contain only a limited number of guests.

For example, if a table has 8 seats:

3. **Family constraint:** Members of certain families must be seated together.
4. **Accessibility constraint:** Guests requiring accessibility must be assigned to appropriate tables.
5. **VIP constraint:** VIP guests must be assigned to tables close to the head table.
6. **Location constraint:** Some guests may require specific table locations because of dietary or other requirements.

The goal is to find an assignment for all guests that satisfies **all constraints simultaneously**.

ii. Backtracking Search

Backtracking is a systematic search technique that assigns values to variables one at a time.

For example:

1. Select Guest 1.
2. Assign Guest 1 to Table 1.
3. Check whether the assignment violates any constraint.
4. Select Guest 2 and assign a possible table.
5. Continue assigning guests.
6. If a constraint is violated, undo the most recent assignment.
7. Try another available table.

8. Continue until all guests have valid table assignments.

For example, if Guest A and Guest B have a conflict and both are assigned to Table 2, the system backtracks and moves Guest B to another valid table.

Thus, backtracking avoids continuing with an arrangement that is already invalid.

iii. Constraint Propagation and Forward Checking

Forward checking improves backtracking by removing invalid choices from the domains of unassigned variables.

Suppose Guest A is assigned to Table 2. If Guest B has a conflict with Guest A, Table 2 is immediately removed from Guest B's domain.

Initially:

After assigning A to :

This prevents the algorithm from trying an assignment that is already known to be invalid.

Forward checking is useful because:

- It detects failures earlier.
- It reduces the search space.
- It decreases unnecessary backtracking.
- It improves the speed of finding a valid arrangement.

For a large wedding, techniques such as **MRV (Minimum Remaining Values)** can also be used to select the most restricted guest first.

iv. Real-World Challenges and Scalable Strategy

Real-world seating arrangements can be difficult because:

- The number of guests may be very large.
- There can be hundreds of constraints.
- Some constraints may conflict with each other.
- Last-minute guest additions or cancellations may occur.
- Table capacities and locations may change.

- Family and social preferences can be difficult to represent mathematically.
- A valid solution may not always exist.

For scalability, the best strategy is to combine **constraint propagation, backtracking, and intelligent variable/value ordering**.

A practical system can use:

CSP + Forward Checking + MRV + Constraint Propagation

For very large events, the problem can also be divided into smaller groups, such as families or VIPs, before performing detailed table assignments.

---

2. Scenario

A fintech startup is developing an AI agent to compete in a simulated auction-bidding game against other automated bidders. The AI must compete against adversarial bidding agents, make decisions within strict time limits per bidding round, evaluate complex market states with multiple possible bids, and the decision tree of possible bid sequences is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem.

ii. Analyze the computational challenges of large game/decision trees.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

iv. Design an evaluation function and justify the factors considered.

v. Recommend a strategy for balancing optimality and real-time performance.

AI Agent for an Adversarial Auction-Bidding Game

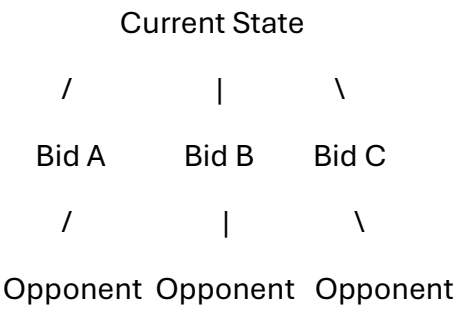
i. Minimax Algorithm

The auction game contains multiple automated bidders competing against each other. Therefore, the AI must consider that its opponent will also make decisions that maximize their own benefit.

Minimax models the game as a decision tree.

- **MAX node:** Represents the AI's turn. The AI selects the action giving the highest expected utility.
- **MIN node:** Represents the opponent's turn. The opponent is assumed to select the action that minimizes the AI's utility.

For example:



The algorithm explores possible sequences of bids and selects the move with the best worst-case outcome.

ii. Computational Challenges

Auction games can produce extremely large decision trees.

If there are possible actions at each level and the search depth is , the approximate number of nodes is:

For example, with 10 possible bids and a depth of 8:

possible nodes may need to be considered.

The major challenges are:

- Huge branching factor.
- Large search depth.
- Limited time per bidding round.
- Complex market states.
- Rapidly changing opponent behavior.
- High memory and processing requirements.

Therefore, searching the entire tree is generally impractical.

iii. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- **Alpha ()** – the best value that MAX can guarantee.
- **Beta ()** – the best value that MIN can guarantee.

When:

the remaining branches do not need to be explored because they cannot improve the final decision.

Example

Suppose the AI has already found a move with a value of 80. While examining another branch, the opponent can force the result to be 60 or less.

Since the AI already has an option worth 80, there is no reason to explore the remaining possibilities of that branch.

Thus, Alpha-Beta pruning:

- Reduces the number of nodes evaluated.
- Allows deeper searches within the same time.
- Produces the **same optimal Minimax decision** when the same search depth is fully evaluated.
- Is especially effective when good moves are examined first.

With ideal move ordering, the effective complexity can improve from approximately:

to:

iv. Evaluation Function

Since the complete game tree cannot usually be searched, the AI needs an **evaluation function** at a selected search depth.

One possible evaluation function is:

where:

- = value of items won.
- = probability of winning future auctions.
- = remaining financial resources.

- = information or strategic advantage.
- = bidding cost/risk.
- = importance weights.

A good evaluation function should consider both **immediate profit and future opportunities**.

For example, spending too much on one item may produce a short-term advantage but leave insufficient resources for later auctions.

v. Strategy for Optimality and Real-Time Performance

A suitable strategy is:

Iterative Deepening Minimax + Alpha-Beta Pruning + Evaluation Function

The AI first searches to a small depth. If time remains, it searches deeper.

For example:

Depth 1 → Depth 2 → Depth 3 → Depth 4 → ...

If the time limit is reached, the AI uses the best result from the deepest completed search.

Additional techniques such as **good move ordering, transposition tables, and adaptive search depth** can further improve performance.

Therefore, this approach provides a good balance between **decision quality, computational efficiency, and strict real-time requirements**.

---

3. Scenario

A marine research team deploys an autonomous underwater vehicle (AUV) to locate survivors near a sunken vessel. The environment is highly dynamic - underwater currents shift debris and visibility, and travel cost changes with depth and current strength. The AUV has access to: a heuristic estimate (straight-line distance to the last known survivor location), partial real-time sonar updates about newly obstructed paths, and variable movement costs depending on current strength and depth. Given the urgency of survivor rescue, the AUV must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

iii. Analyze the impact of heuristic accuracy on both algorithms.

iv. Discuss how dynamic changes (shifting debris and visibility) affect search

performance.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Autonomous Underwater Vehicle Route Planning

i. Greedy Best-First Search

Greedy Best-First Search selects the node that appears closest to the goal according to the heuristic.

Its evaluation function is:

where is the estimated distance from the current position to the survivor's location.

In this scenario, the AUV would generally select the path that appears to move most directly toward the last known survivor location.

Strengths

- Fast and simple.
- Usually explores fewer nodes.
- Useful when the goal location is approximately known.
- Suitable when rapid decisions are important.

Weaknesses

- It ignores the cost already spent reaching a node.
- The shortest-looking path may have strong currents or obstacles.
- It may choose a dangerous route.
- It does not guarantee the minimum-cost path.
- An inaccurate survivor location can mislead the search.

Therefore, Greedy Best-First Search prioritizes **speed over guaranteed optimality**.

ii. A* Search

A* combines the actual cost of reaching a node with the estimated cost to the goal.

Its evaluation function is:

where:

- = actual cost from the starting point to node .
- = estimated cost from to the goal.

For the AUV:

- can include travel time, energy consumption, current strength, depth cost, and risk.
- can estimate the remaining distance or expected travel cost.

For example, a path that looks slightly longer but has weak currents may have a lower total cost than a shorter path through a strong current.

Therefore, A* balances **the cost already incurred and the estimated remaining cost**.

iii. Impact of Heuristic Accuracy

The quality of the heuristic strongly affects both algorithms.

Greedy Best-First Search

Greedy search depends heavily on .

If the heuristic is accurate, it can quickly move toward the correct area.

If the heuristic is inaccurate because the survivor has moved or the last known location is wrong, Greedy Search may follow an inefficient route.

A*

A* is also affected by heuristic accuracy, but it considers both:

Therefore, the actual travel cost can prevent it from blindly following a misleading heuristic.

If the heuristic is **admissible**, meaning it does not overestimate the true remaining cost, A* can provide an optimal solution in a static environment.

However, in a changing underwater environment, even a good heuristic can become outdated.

iv. Effect of Dynamic Changes

The underwater environment can change while the AUV is searching.

Examples include:

- Debris blocking a previously available path.
- Visibility changing.
- Currents becoming stronger or weaker.
- Travel costs changing with depth.
- New sonar information revealing obstacles.
- The survivor's location changing.

These changes can make a previously calculated path invalid or expensive.

For example, the AUV may calculate:

but new sonar information may reveal that is blocked.

The search must then update its path and find another route.

Repeatedly running a traditional search from the beginning may waste time. More suitable approaches for highly dynamic environments include **incremental or replanning versions of A***.

v. Recommended Algorithm

For this scenario, **A*** is generally more suitable than Greedy Best-First Search because the AUV must consider not only distance but also **travel cost, current strength, depth, obstacles, and safety**.

A practical strategy would be:

*A + real-time sonar updates + frequent replanning**

A* provides a better balance between:

- **Speed:** Uses a heuristic to guide the search.
- **Optimality:** Considers both and .

- **Safety:** Can incorporate risk and obstacle costs.
- **Adaptability:** Can recalculate routes when new sonar information arrives.

If the environment changes very frequently, an **incremental A*** approach would be even more appropriate because it can reuse information from previous searches rather than calculating everything from scratch.

Comparison

Factor	**Greedy Best-First Search**	**A***
Evaluation	f(n)=h(n)	f(n)=g(n)+h(n)
Speed	Generally faster	Generally more computationally expensive
Path cost	Ignores previous cost	Considers previous cost
Optimality	Not guaranteed	Can be optimal with suitable heuristic in static conditions
Dynamic environment	Can react but may choose poor paths	Better cost-aware replanning
Safety consideration	Limited unless built into heuristic	Can incorporate cost/risk
Suitability for AUV	Useful for emergency approximate routing	**Better overall choice**
